

A05-Iterators

COMP301

1 Introduction

Picture yourself opening a ride-share app like Uber or Lyft. The app automatically determines your location and searches for drivers who are currently available in your area. Your task in this assignment is to build three classes that implement the `Iterator<>` interface. These iterators will allow you to step through the available drivers logged into the app, one by one, in a specific sequence, using the `next()` and `hasNext()` methods.

- For **Apprentice**, the iterator that you create will act as a filter, identifying and iterating through only the drivers that are near the customer.
- For **Enchanter**, your iterator will search for nearby drivers using an increasing search range; the closest drivers will be iterated through first, and drivers which are farther away will be iterated through later.
- Finally, for **Sorcerer**, your iterator will interleave multiple collections of drivers together, providing a single iterator that goes through the drivers in each of the collections.

To get started, read through the code already provided. The interfaces `Position`, `Driver`, and `Vehicle`, and their implementations `PositionImpl`, `DriverImpl`, and `VehicleImpl`, are complete, and you should not have to modify them in any way for this assignment.

1.1 Interfaces

Here is a summary of the three provided interfaces.

1. The `Driver` interface represents a driver working for a ride sharing service like Uber or Lyft. Every driver is assigned a vehicle that they use to offer rides to their customers.
2. The `Vehicle` interface represents a vehicle which can be used by a particular driver. Every vehicle is assigned a position, which indicates the current GPS location of the vehicle.
3. The `Position` interface represents the GPS location of a vehicle. For the purpose of this assignment, a position will be represented by an (x, y) .

1.2 Distance calculation

The `Position` interface has a default method called `getManhattanDistanceTo(Position p)`. The purpose of this method is to calculate the Manhattan distance from the location to another `Position` object, p . The Manhattan distance between two points (x_1, y_1) and (x_2, y_2) is defined as:

$$d_{\text{Manhattan}} = |x_1 - x_2| + |y_1 - y_2|$$

The idea is that if the ride sharing application operates in a city with gridded streets (like Manhattan), it would be impossible to travel straight from one position to another, due to the buildings in the way. Instead, the driver would have to travel along the gridded streets. All distance calculations for this assignment will therefore use the `getManhattanDistanceTo()` method.

1.3 driverpools

The collection of `Driver` objects to iterate through represents the pool of drivers currently available on the app. Generally, the pool of available drivers will be passed into the constructor of your iterator class in the form of an `Iterable<Driver>` object. Your iterator class will use this object to access the available `Driver` objects.

How can you use an `Iterable<Driver>` object to access the available drivers? One way is to call the `iterator()` method, which creates and returns an iterator object. You can then call `next()` on that iterator object repeatedly to get access to the `Driver` objects.

In summary, each of your iterator classes for this assignment will implement the `Iterator<Driver>` interface, but they will also encapsulate another instance of `Iterator<Driver>`. Your iterator class will use the encapsulated iterator to access the `Driver` objects. It will rearrange or filter the `Driver` objects, and finally will return them in `next()` one at a time, in the new order.

2 Apprentice (10 points)

The first iterator class to create for this assignment will act as a filter on the pool of available `Driver` objects. It will not modify the order of the drivers; instead, it will only iterate over the drivers which are close enough to the customer's position.

Create a class called `ProximityIterator` that implements the interface `Iterator<Driver>`. You'll have to add the `next()` and `hasNext()` methods to the class in fulfillment of the interface requirements. The constructor for the class should be declared with the following signature.

Listing 1: ProximityIterator Constructor Signature

```
1 public ProximityIterator(Iterable<Driver> driverPool, Position clientPosition,
   int proximityRange)
```

Here is a description of the constructor's parameters:

- `driverPool` - Represents the available pool of `Driver` objects. Call `driverPool.iterator()` to receive an iterator object for the underlying collection of available drivers.
- `clientPosition` - Represents the static position of the customer/client.
- `proximityRange` - Represents the range to use when deciding whether to include a `Driver` object in the iterator. Only `Driver` objects located within `proximityRange` units from `clientPosition` should be included by your iterator.

Your `ProximityIterator` class should iterate only through the `Driver` objects in the collection that have a Manhattan distance to `clientPosition` that is less than or equal to `proximityRange`. If `next()` is called but there are no more eligible drivers, throw a `NoSuchElementException`

2.1 Tips for Apprentice

- In your constructor, use the `iterator()` method of `driverPool` to create an iterator for all of the `Driver` objects in the collection. Store this iterator in an instance field.
- Use an instance field to store the next driver that is within the proximity range from the client. Initialize this to `null` in your constructor.
- Add a `private` instance method which loads the next driver that is within the proximity range into the "next driver" field. If the "next driver" field is already set (i.e., if it is already non-`null`), this method should do nothing. Otherwise, it should retrieve drivers from the driverpool iterator until either an appropriate driver is found, or until no more drivers are in the pool. If an appropriate driver is found, store it in the "next driver" instance field. Otherwise, leave the "next driver" field `null`, because no next driver exists.
- To implement `hasNext()`, first make sure that a driver has been loaded into the "next driver" field, by calling your `private` method. Then, if a driver was successfully loaded in, return `true`; otherwise, return `false`.

- To implement `next()`, first call `hasNext()`. If `false`, throw a `NoSuchElementException`. If `true`, then you know that the next eligible driver must be in your “next driver” instance field. Copy this to a local variable. Reset your “next driver” field to be `null` so that you don’t keep returning the same driver. Now return the value of the local variable that is storing the next driver.

3 Enchanter (5 points)

The second iterator class to create for this assignment will scan through the pool of `Driver` objects multiple times, using a gradually increasing search range each time. The first time through the driverpool, only drivers within 1 unit from the client will be selected. The second time through the driverpool, the search range for finding drivers will increase by a configurable amount. The third time, the search range will increase again. This process continues until all drivers in the pool are visited.

Create a class called `ExpandingProximityIterator` that implements the interface `Iterator<Driver>`. You’ll have to add the `next()` and `hasNext()` methods to the class in fulfillment of the interface requirements. The constructor for the class should be declared with the following signature.

Listing 2: `ExpandingProximityIterator` Constructor Signature

```
1 public ExpandingProximityIterator(Iterable<Driver> driverPool, Position
   clientPosition, int expansionStep)
```

Here is a description of the constructor’s parameters:

- `driverPool` - Represents the available pool of `Driver` objects. Call `driverPool.iterator()` to receive an iterator object for the underlying collection of available drivers.
- `clientdriverPosition` - Represents the static position of the customer/client.
- `expansionStep` - Represents the amount by which to increase the range when searching for available drivers.

An `ExpandingProximityIterator` should first iterate through all of the `Driver` objects in the collection that have a Manhattan distance to `clientPosition` that is less than or equal to 1. After all such drivers have been exhausted, the iterator should start back at the beginning of the pool and iterate through drivers that have a distance that is > 1 but $\leq 1 + \text{expansionStep}$. After all of these drivers have been exhausted, the iterator should start at the beginning of the pool again and iterate through drivers that have a distance that is $> 1 + \text{expansionStep}$ but $\leq 1 + 2 \times \text{expansionStep}$. This process should continue, with the range limit for the k -th pass being $1 + k \times \text{expansionStep}$, until there are no more drivers left to process. At that point, `hasNext()` should return `false`.

3.1 Tips for Enchanter

- You will need to encapsulate `driverPool` as an instance field, so that you can use it to create a new iterator each time you run out of drivers for a particular “ring” size.
- You will need to detect when you have run out of drivers completely so that `hasNext()` returns `false` eventually. There are a couple of different ways to do this.
 1. One way is to maintain a `boolean` flag that is set whenever a driver is encountered that is outside of the current ring while looking for the next driver. Then, when you have run out of drivers at the current ring size, you can check this flag to make sure that at least one driver is still outside of the current ring, and you know if you should keep expanding. Be sure to reset the flag each time you expand.
 2. Another way to do this is to keep track of the total number of drivers in the driverpool. Then, keep track of the number of times `next()` has been successfully called. When this count is equal to the size of the driverpool, you know that every driver has been found at some ring size already and that expanding the ring won’t help (i.e., there are no more drivers).

4 Sorcerer (5 points)

The last iterator class to create for this assignment will interleave the drivers from multiple different driverpools together into one iterator.

Create a class called `SnakeOrderAcrossPoolsIterator` that implements the interface `Iterator<Driver>`. The constructor for this class should be declared with the following signature:

Listing 3: SnakeOrderAcrossPoolsIterator Constructor Signature

```
1 public SnakeOrderAcrossPoolsIterator(List<Iterable<Driver>> driverPools)
```

Unlike Apprentice and Enchanter, the constructor of this iterator is given a *list* of driverpools. This version of the iterator should retrieve the next driver from each pool, one at a time, in “snake order”. “Snake order” means first going from first-to-last and then reversing and going from last-to-first. For example, let’s say there are 4 driverpools in the `driverPools` list. The sequence of pool indices from which to retrieve the next driver would be:

$$0, 1, 2, 3, \underbrace{3, 2, 1, 0}_{\text{reverse}}, \underbrace{0, 1, 2, 3}_{\text{forward}}, \dots$$

The pools may have a different number of drivers in them. Once a pool runs out of drivers, it is simply skipped over. This iterator will run out of drivers once all the pools in the list have run out. At that point, `hasNext()` should return `false`.

4.1 Tips for Sorcerer

- In your constructor, use the list of driverpools provided to create a corresponding list of driver **iterators** for each pool; store this list as an instance field .
- Encapsulate an integer instance field to store an index representing which pool iterator to pull from the next time `next()` is called.

5 Grading

This assignment is worth 20 points.

- Apprentice: 10 points
- Enchanter: 5 points
- Sorcerer: 5 points